ЛАБОРАТОРНАЯ РАБОТА 5. Лямбда-выражеиян

Цель работы: Создать java-проекте с использованием лямбда-выражений

Общие сведения

Лямбда представляет набор инструкций, которые можно выделить в отдельную переменную и затем многократно вызвать в различных местах программы.

Основу лямбда-выражения составляет *лямбда-оператор*, который представляет стрелку ->. Этот оператор разделяет лямбда-выражение на две части: левая часть содержит список параметров выражения, а правая собственно представляет тело лямбда-выражения, где выполняются все действия.

Лямбда-выражение не выполняется само по себе, а образует реализацию метода, определенного в *функциональном интерфейсе*. При этом важно, что функциональный интерфейс должен содержать только один единственный метод без реализации.

Рассмотрим пример:

```
public class LambdaApp {
    public static void main(String[] args) {
        Operationable operation;
        operation = (x,y)->x+y;
        int result = operation.calculate(10, 20);
        System.out.println(result); //30
    }
}

public interface Operationable {
    int calculate(int x, int y);
}
```

В роли функционального интерфейса выступает интерфейс Operationable, в котором определен один метод без реализации – метод calculate. Данный метод принимает два параметра - целых числа, и возвраща-

ет некоторое целое число.

По факту лямбда-выражения являются в некотором роде сокращенной формой *внутренних анонимных классов*, которые ранее применялись в Java. В частности, предыдущий пример мы можем переписать следующим образом, изменив класс LambdaApp при неизменном интерфейсе Operationable.

```
public class LambdaApp {
    public static void main(String[] args) {

        Operationable op = new Operationable(){
            public int calculate(int x, int y){
                return x + y;
            }
        };
        int z = op.calculate(20, 10);
        System.out.println(z); // 30
    }
}
```

Чтобы объявить и использовать лямбда-выражение, основная программа разбивается на ряд этапов:

1. Определение ссылки на функциональный интерфейс:

```
Operationable operation;
```

2. Создание лямбда-выражения:

```
operation = (x,y)->x+y;
```

Причем параметры лямбда-выражения соответствуют параметрам единственного метода интерфейса Operationable, а результат соответствует возвращаемому результату метода интерфейса. При этом нам не надо использовать ключевое слово return для возврата результата из лямбда-выражения.

Так, в методе интерфейса оба параметра представляют тип int, значит, в теле лямбда-выражения мы можем применить к ним сложение. Результат

сложения также представляет тип int, объект которого возвращается методом интерфейса.

3. Использование лямбда-выражения в виде вызова метода интерфейса:

```
int  result = operation.calculate(10, 20);
```

Так как в лямбда-выражении определена операция сложения параметров, результатом метода будет сумма чисел 10 и 20.

При этом для одного функционального интерфейса мы можем определить множество лямбда-выражений. Например:

```
Operationable operation1 = (int x, int y)-> x + y;
Operationable operation2 = (int x, int y)-> x - y;
Operationable operation3 = (int x, int y)-> x * y;

System.out.println(operation1.calculate(20, 10)); //30
System.out.println(operation2.calculate(20, 10)); //10
System.out.println(operation3.calculate(20, 10)); //200
```

Отложенное выполнение

Одним из ключевых моментов в использовании лямбд является отложенное выполнение (deferred execution). То есть мы определяем в одном месте программы лямбда-выражение и затем можем его вызывать при необходимости неопределенное количество раз в различных частях программы. Отложенное выполнение может потребоваться, к примеру, в следующих случаях:

- выполнение кода отдельном потоке;
- выполнение одного и того же кода несколько раз;
- выполнение кода в результате какого-то события;
- выполнение кода только в том случае, когда он действительно необходим и, если он необходим.

Передача параметро в лямбда-выражение

Параметры лямбда-выражения должны соответствовать по типу параметрам метода из функционального интерфейса. При написании самого лямбда-выражения тип параметров писать необязательно, хотя в принципе это можно сделать, например:

```
operation = (int x, int y)->x+y;
```

Если метод не принимает никаких параметров, то пишутся пустые скобки, например:

```
( )-> 30  + 20;
```

Если метод принимает только один параметр, то скобки можно опустить:

```
n-> n * n;
```

Терминальные лямбда-выражения

Выше мы рассмотрели лямбда-выражения, которые возвращают определенное значение. Но также могут быть и терминальные лямбды, которые не возвращают никакого значения. Например:

```
interface Printable{
    void print(String s);
}

public class LambdaApp {

    public static void main(String[] args) {

        Printable printer = s->System.out.println(s);
        printer.print("Hello Java!");
    }
}
```

Лямбда-выражение и локальные переменные

Лямбда-выражение может использовать переменные, которые объявлены во вне в более общей области видимости – на уровне класса или метода, в котором лямбда-выражение определено. Однако в зависимости от того, как и где определены переменные, могут различаться способы их использования в лямбдах. Рассмотрим первый пример – использования переменных уровня класса:

```
public class LambdaApp {

    static int x = 10;
    static int y = 20;

    public static void main(String[] args) {

            Operation op = ()->{
                x=30;
                return x+y;
            };
            System.out.println(op.calculate()); // 50
            System.out.println(x); // 30 - значение x изменилось
    }
}
```

Результат выполнения программы приведен на рис. 1

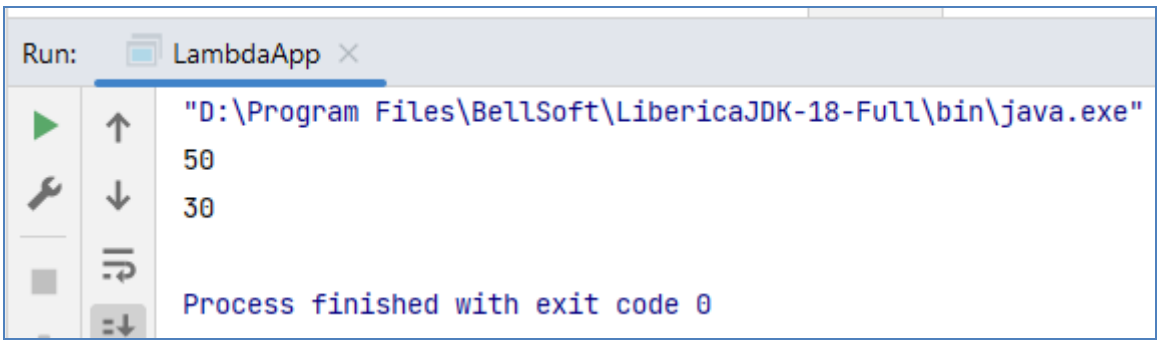


Рисунок 1 – Результат использования переменных уровня класса

Переменные x и y объявлены на уровне класса, и в лямбда-выражении мы их можем получить и даже изменить. Так, в данном случае после выполнения выражения изменяется значение переменной x.

Теперь рассмотрим другой пример – локальные переменные на уровне метода:

```
public class LambdaApp {

    public static void main(String[] args) {

        int n=70;
        int m=30;
        Operation op = ()->{

            //n=100; - так нельзя сделать
            return m+n;
        };
        // n=100;  - так тоже нельзя
        System.out.println(op.calculate()); // 100
    }
}
```

Локальные переменные уровня метода мы также можем использовать в лямбдах, но изменять их значение нельзя. Если мы попробуем это сделать, то среда разработки (Netbeans или другая) может нам высветить ошибку и то, что такую переменную надо пометить с помощью ключевого слова final, то есть сделать константой: final int n=70;. Однако это необязательно.

Более того, мы не сможем изменить значение переменной, которая используется в лямбда-выражении, вне этого выражения. То есть даже если такая переменная не объявлена как константа, по сути она является константой.

Блоки кода в лямбда-выражениях

Существуют два типа лямбда-выражений: однострочное выражение и блок кода. Примеры однострочных выражений демонстрировались выше. Блочные выражения обрамляются фигурными скобками. В блочных лямбда-выражениях можно использовать внутренние вложенные блоки, циклы, конструкции if, switch, создавать переменные и т.д. Если блочное лямбда-выражение должно возвращать значение, то явным образом применяется оператор return:

```
public class LambdaApp {
    static int x = 10;
    static int y = 20;
    public static void main(String[] args) {
        Operationable operation = (int x, int y)-> {
            if(y==0)
                return 0;
```

```
            else                                                return x/y;
        };
        System.out.println(operation.calculate(20, 10)); //2
        System.out.println(operation.calculate(20, 0)); //0
    }
}
```

Обобщенный функциональный интерфейс

Функциональный интерфейс может быть *обобщенным*, однако в лямбда-выражении использование обобщений не допускается. В этом случае нам надо типизировать объект интерфейса определенным типом, который потом будет применяться в лямбда-выражении. Например:

```
public class LambdaApp {
    public static void main(String[] args) {
        Operationable<Integer> operation1 = (x, y)-> x + y;
        Operationable<String> operation2 = (x, y) -> x + y;
        System.out.println(operation1.calculate(20, 10)); //30
        System.out.println(operation2.calculate("20", "10")); //2010
    }
}

public interface Operationable<T> {
    T calculate(T x, T y);
}
```

Результат выполнения программы приведен на рис.2

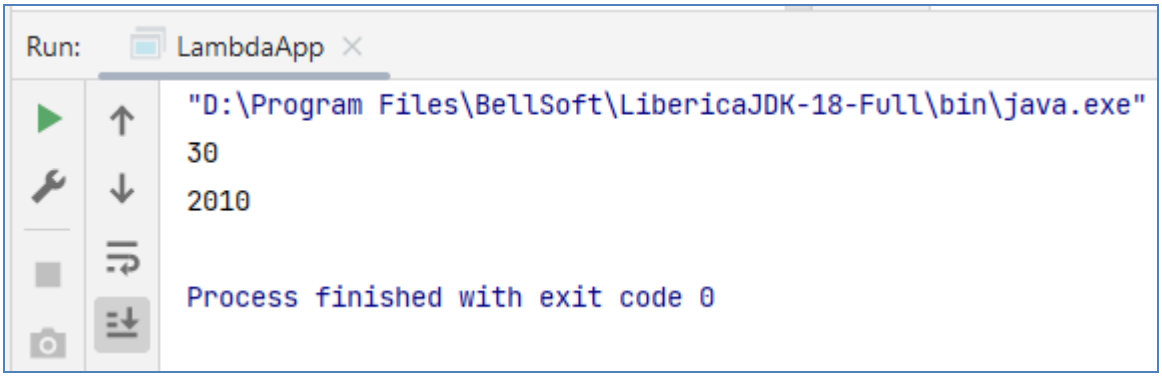


Рисунок 2 – Результат использования обобщенного функционального интерфейса

Таким образом, при объявлении лямбда-выражения ему уже известно, какой тип параметры будут представлять и какой тип они будут возвращать.

Лямбды как параметры и результаты методов

Лямбды как параметры методов

Одним из преимуществ лямбда-выражений в java является то, что их можно передавать в качестве параметров в методы. Рассмотрим пример:

```
public class Main {
    public static void main(String[] args) {

        Expression func = (n)-> n%2==0;
        int[] nums = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
        System.out.println(sum(nums, func)); // 20
    }
    private static int sum (int[] numbers, Expression func)
    {
        int result = 0;
        for(int i : numbers)
        {
            if (func.isEqual(i))
                result += i;
        }
        return result;
    }
}
```

Функциональный интерфейс Expression определяет метод isEqual( ), который возвращает true, если в отношении числа n действует какое-нибудь равенство.

В основном классе программы определяется метод sum( ), который вычисляет сумму всех элементов массива, соответствующих некоторому условию. А само условие передается через параметр Expression func. Причем на момент написания метода sum мы можем абсолютно не знать, какое именно условие будет использоваться. Само же условие определяется в виде лямбда-выражения:

```
Expression func = (n)-> n%2==0;
```

То есть в данном случае все числа должны быть четными или остаток от их деления на 2 должен быть равен 0. Затем это лямбда-выражение передается в вызов метода sum.

При этом можно не определять переменную интерфейса, а сразу пере-

дать в метод лямбда-выражение:

```
int[] nums = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
int x = sum(nums, (n)-> n > 5); // сумма чисел, которые больше 5
System.out.println(x);  // 30
```

Ссылки на метод как параметры методов

Начиная с JDK 8 в Java можно в качестве параметра в метод передавать ссылку на другой метод. В принципе данный способ аналогичен передаче в метод лямбда-выражения.

Ссылка на метод передается в виде *имя_класса::имя_статического_метода* (если метод статический) или *объект_класса::имя_метода* (если метод нестатический). Рассмотрим на примере:

```
public class Main {
    public static void main(String[] args) {

    int[] nums = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    System.out.println(sum(nums, ExpressionHelper::isEven));

    Expression expr = ExpressionHelper::isPositive;
    System.out.println(sum(nums, expr));
  }

    private static int sum (int[] numbers, Expression func)
    {
        int result = 0;
        for(int i : numbers)
        {
            if (func.isEqual(i))
                result += i;
        }
        return result;
    }
}
public interface Expression {
    boolean isEqual(int n);
}

public class ExpressionHelper {
    static boolean isEven(int n){
        return n%2 == 0;
    }

    static boolean isPositive(int n){
        return n > 0;
    }
}
```

Здесь также определен функциональный интерфейс Expression, который имеет один метод. Кроме того, определен класс ExpressionHelper, который содержит два статических метода: isEven и isPositive. В принципе их можно было определить и в основном классе программы, но я вынес их в отдельный класс.

В основном классе программы LambdaApp определен метод sum(), который возвращает сумму элементов массива, соответствующих некоторому условию. Условие передается в виде объекта функционального интерфейса Expression.

В методе main два раза вызываем метод sum, передавая в него один и тот же массив чисел, но разные условия. Первый вызов метода sum:

System.out.println(sum(nums, ExpressionHelper::isEven));

На место второго параметра передается ExpressionHelper::isEven, то есть ссылка на статический метод isEven() класса ExpressionHelper. При этом методы, на которые идет ссылка, должны совпадать по параметрам и результату с методом функционального интерфейса.

При втором вызове метода sum отдельно создается объект Expression, который затем передается в метод:

```
Expression expr = ExpressionHelper::isPositive;
System.out.println(sum(nums, expr));
```

Использование ссылок на методы в качестве параметров аналогично использованию лямбда-выражений.

Если нам надо вызвать нестатические методы, то в ссылке вместо имени класса применяется имя объекта этого класса:

```
interface Expression{
    boolean isEqual(int n);
}
```

```
class ExpressionHelper{
    boolean isEven(int n){
    return n%2 == 0;
    }
}
public class LambdaApp {

    public static void main(String[] args) {

        int[] nums = { -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5};
        ExpressionHelper exprHelper = new ExpressionHelper();
        System.out.println(sum(nums, exprHelper::isEven)); // 0
    }

    private static int sum (int[] numbers, Expression func)
    {
        int result = 0;
        for(int i : numbers)
        {
            if (func.isEqual(i))
                result += i;
        }
        return result;
    }
}
```

Ссылки на конструкторы

Подобным образом мы можем использовать конструкторы: *название_класса::new*. Например:

```
public class LambdaApp {

    public static void main(String[] args) {

        UserBuilder userBuilder = User::new;
        User user = userBuilder.create("Tom");
        System.out.println(user.getName());
    }
}
interface UserBuilder{
    User create(String name);
}
```

```
class User{

    private String name;
    String getName(){
        return name;
    }

    User(String n){
        this.name=n;
    }
}
```

При использовании конструкторов методы функциональных интерфейсов должны принимать тот же список параметров, что и конструкторы класса, и должны возвращать объект данного класса.

Лямбды как результат методов

Также метод в Java может возвращать лямбда-выражение. Рассмотрим следующий пример:

```
interface Operation{
    int execute(int x, int y);
}

public class LambdaApp {

    public static void main(String[] args) {

        Operation func = action(1);
        int a = func.execute(6, 5);
        System.out.println(a);          // 11

        int b = action(2).execute(8, 2);
        System.out.println(b);          // 6
    }

    private static Operation action(int number){
        switch(number){
            case 1: return (x, y) -> x + y;
            case 2: return (x, y) -> x - y;
            case 3: return (x, y) -> x * y;
            default: return (x,y) -> 0;
```

```
        }
    }
}
```

В данном случае определен функциональный интерфейс Operation, в котором метод execute принимает два значения типа int и возвращает значение типа int.

Метод action принимает в качестве параметра число и в зависимости от его значения возвращает то или иное лямбда-выражение. Оно может представлять либо сложение, либо вычитание, либо умножение, либо просто возвращает 0. Стоит учитывать, что формально возвращаемым типом метода action является интерфейс Operation, а возвращаемое лямбда-выражение должно соответствовать этому интерфейсу.

В методе main мы можем вызвать этот метод action. Например, сначала получить его результат - лямбда-выражение, которое присваивается переменной Operation. А затем через метод execute выполнить это лямбда-выражение:

```
Operation func = action(1);
int a = func.execute(6, 5);
System.out.println(a);          // 11
```

Либо можно сразу получить и тут же выполнить лямбда-выражение:

```
int b = action(2).execute(8, 2);
System.out.println(b);          // 6
```

Задание на лабораторную работу

1. Изучить теоретический материал
2. Создать приложение, реализующее лямбда-выражения в соответствии с описанием, приведенным в лабораторной работе.
3. Разработать приложение с использованием лямбда-выражений, по заданию преподавателя.
4. Продемонстрировать преподавателю работу созданного приложения.

5. Подготовить отчет по лабораторной работе, загрузить отчет в систему Moodle, защитить отчет.